5 Techniques

Reranking in RAG

1. Cross-Encoder Reranking
2. Reciprocal Rank Fusion (RRF)
3. Cohere Rerank API
4. ColBERT (Late Interaction)
5. LLM-as-a-Judge

From fast retrieval to accurate context: how reranking sharpens your final answer.

HRUTHIK GALI DILEEP

FOLLOW

Repost

RERANKING

What is Reranking?

Reranking is a second-stage refinement process in RAG pipelines that reorders retrieved documents based on their true relevance to the query. While initial retrieval methods (like vector search or BM25) quickly fetch candidate documents, reranking uses more sophisticated models to deeply analyze the semantic relationship between the query and each document, surfacing the most contextually relevant results.

Why Reranking Matters in RAG

Initial retrieval methods prioritize speed and often use simple similarity metrics, which can lead to suboptimal results. Reranking addresses critical challenges:

- Improved Precision: Ensures the most relevant chunks reach the LLM, leading to more accurate answers
- Noise Reduction: Filters out semantically similar but contextually irrelevant documents
- Better Context Quality: Provides focused, high-quality context within token limits
- Enhanced User Experience: Reduces hallucinations and improves answer relevance
- Cost Efficiency: Fewer irrelevant tokens processed by expensive LLMs

Top 5 Reranking Techniques for RAG

1. **Cross-Encoder Reranking** - Deep neural scoring of query-document pairs
2. **Reciprocal Rank Fusion (RRF)** - Combines rankings from multiple retrieval methods
3. **Cohere Rerank API** - Managed reranking service with state-of-the-art models
4. **ColBERT (Late Interaction)** - Efficient token-level similarity matching
5. **LLM-as-a-Judge** - Uses large language models to evaluate relevance

Cross-Encoder Reranking

What is Cross-Encoder Reranking?

Cross-Encoder reranking uses transformer models that process the query and document together as a concatenated input, allowing deep token-level interactions. Unlike bi-encoders that encode query and documents separately, cross-encoders capture fine-grained semantic relationships and output a relevance score for each query-document pair.

When to Use

- When accuracy is more important than speed
- For reranking top-k candidates (typically 20-100 documents)
- When you need nuanced understanding of query-document relevance
- In production RAG systems where quality directly impacts user satisfaction

How to Use

Use pre-trained models like ms-marco-MiniLM-L-6-v2 or cross-encoder/ms-marco-MiniLM-L-12-v2 from the sentence-transformers library. Pass query-document pairs through the model to get relevance scores, then sort by score.

Use Case

Technical Documentation Search: A developer searches "how to handle authentication errors in API." Initial retrieval returns 50 documents about APIs, authentication, and errors. Cross-encoder reranking identifies that the document specifically covering "error handling for failed authentication requests" is most relevant, even if other documents have similar keywords.

Reciprocal Rank Fusion (RRF)

What is Reciprocal Rank Fusion?

RRF is a technique that combines rankings from multiple retrieval methods (e.g., BM25 keyword search + vector similarity search) into a single unified ranking. It uses the reciprocal of each document's rank position across different methods to compute a final score, giving higher weight to documents that rank well across multiple systems.

Formula: RRF_score = Σ 1/(k + rank_i) where k is a constant (typically 60)

When to Use

- When you have multiple retrieval methods available
- To leverage both semantic and keyword-based search strengths
- When you want a simple, training-free reranking approach
- For systems where different queries benefit from different retrieval types

How to Use

Run multiple retrieval methods in parallel, collect their rankings, apply the RRF formula to each document across all rankings, then sort by the combined RRF score.

Use Case

E-commerce Product Search: A user searches for "wireless noise cancelling headphones under $200." BM25 retrieval finds products with exact keyword matches, while vector search finds semantically similar products (like "bluetooth ANC earbuds"). RRF combines both rankings to surface products that match both the semantic intent and specific keywords, improving relevance.

Cohere Rerank API

What is Cohere Rerank?

Cohere Rerank is a managed API service that provides state-of-the-art reranking capabilities without requiring infrastructure setup or model deployment. It uses advanced transformer models trained on diverse datasets to score query-document relevance and can handle multilingual content.

When to Use

- When you want production-ready reranking without infrastructure overhead
- For rapid prototyping and experimentation
- When you need multilingual reranking support
- If you prefer managed services over self-hosted models

How to Use

Call the Cohere API with your query and list of documents. The API returns relevance scores for each document. You can set a top-n parameter to get only the most relevant results.

```
import cohere
co = cohere.Client('YOUR_API_KEY')
results = co.rerank( query="quantum computing applications", documents=retrieved_chunks,
top_n=5, model='rerank-english-v2.0')
```

Use Case

News Aggregation Platform: A news app needs to rerank articles from multiple sources for personalized feeds. Using Cohere Rerank API, the system quickly scores hundreds of articles against user interests without maintaining reranking infrastructure, enabling real-time personalization at scale.

ColBERT (Late Interaction)

What is ColBERT?

ColBERT (Contextualized Late Interaction over BERT) computes fine-grained token-level interactions between query and document embeddings efficiently. Unlike cross-encoders that process pairs jointly or bi-encoders that compute single vectors, ColBERT creates embeddings for each token and performs late interaction through maximum similarity matching.

When to Use

- When you need a balance between bi-encoder speed and cross-encoder accuracy
- For large-scale retrieval with better precision than bi-encoders
- When you can precompute and index document token embeddings
- In systems requiring sub-second latency with high relevance

How to Use

Use the RAGatouille library or Stanford's ColBERT implementation. Index your documents to create token-level embeddings, then at query time, compute query token embeddings and perform MaxSim operations against indexed document embeddings.

```
from ragatouille import RAGPretrainedModel
RAG = RAGPretrainedModel.from_pretrained("colbert-ir/colbertv2.0")
RAG.index(documents, index_name="my_index")
results = RAG.search(query="machine learning basics", k=5)
```

Use Case

Academic Paper Search: Researchers searching for papers on "transformer attention mechanisms for long sequences" need precise matching. ColBERT's token-level interactions identify papers that discuss both "attention mechanisms" AND "long sequences" together, rather than papers that only mention these concepts separately, providing better precision than simple vector search.

LLM-as-a-Judge Reranking

What is LLM-as-a-Judge?

LLM-as-a-Judge reranking uses large language models (GPT-4, Claude, etc.) to evaluate and score the relevance of retrieved documents to a query. The LLM acts as an intelligent judge, using its reasoning capabilities to assess how well each document answers the query, often with detailed explanations.

When to Use

- When maximum accuracy is required regardless of cost
- For complex queries requiring reasoning and context understanding
- When you need explainable reranking decisions
- In low-volume, high-value scenarios (medical, legal, financial)

How to Use

Prompt the LLM with the query and each document, asking it to score relevance (e.g., 1-10) or directly rank documents. You can also ask for reasoning to understand why certain documents were ranked higher.

```
prompt = f"""Rate the relevance of this document to the query on a scale of 1-10.
Query: {query}Document: {document}Relevance Score:"""
```

Use Case

Medical Diagnosis Support: A doctor queries "treatment options for stage 2 hypertension in diabetic patients with kidney disease." An LLM judge can reason about the complex multi-condition scenario and identify that a document discussing "combined hypertension-diabetes management with renal considerations" is more relevant than separate documents on each condition, something simpler rerankers might miss.

Implementation Example - Cross-Encoder

```python
from sentence_transformers import CrossEncoder

# 1. Load a reranking model
# You can use others like: "cross-encoder/ms-marco-MiniLM-L-6-v2"
model = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")

# 2. Query (two lines)
query = (
    "Explain how self-attention works in transformer models.\n"
    "Focus on how tokens interact and how weights are computed."
)

# 3. Retrieved context chunks (all related to the query)
retrieved_chunks = [
    "Chunk A: Self-attention allows each token to compare itself to every other token "
    "using similarity scores derived from query, key and value vectors.",

    "Chunk B: In transformers, attention weights are computed by taking the dot product "
    "between query and key vectors, followed by softmax normalization.",

    "Chunk C: Multi-head attention runs multiple attention mechanisms in parallel, "
    "letting the model capture different relationships between tokens.",

    "Chunk D: Positional encodings inject order information into tokens so attention "
    "can understand sequence structure before computing interactions.",

    "Chunk E: Scaled dot-product attention divides the dot product by the square root "
    "of the dimension to stabilize gradients and prevent overly large scores."
]

# 4. Prepare (query, chunk) pairs for scoring
pairs = [(query, chunk) for chunk in retrieved_chunks]

# 5. Get reranking scores from the model
scores = model.predict(pairs)  # Returns a score for each pair

# 6. Attach scores to chunks
scored = [{"chunk": chunk, "score": float(score)}
          for chunk, score in zip(retrieved_chunks, scores)]

# 7. Apply threshold ≥ 8
threshold = 1
filtered = [x for x in scored if x["score"] ⩾ threshold]

# 8. Sort in descending order
top3 = sorted(filtered, key=lambda x: x["score"], reverse=True)[:3]

# 9. Final context
final_context = [x["chunk"] for x in top3]

print("Top 3 reranked chunks:")
for x in top3:
    print(f"{x['score']:.2f} → {x['chunk']}")
```